

Arduino-Based Ultrasonic Radar with Real-Time Processing Visualization

Name: Ritiz Shrestha
Organization/Club: SERN
Post: Founding Member
Date: 17th August 2026

1. Abstract

This project presents an Arduino-based ultrasonic radar system for detecting objects at different angular positions and displaying the measurements in real time. An HC-SR04 ultrasonic sensor was mounted on a servo motor so that it could scan through a range of angles.

The Arduino controls the servo and takes distance measurements using the HC-SR04 at different angular positions. The measured angle and distance are sent to a computer through serial communication. A Processing program was developed to display the measurements as a radar interface. The measured angle and distance are converted from polar coordinates into Cartesian coordinates to determine the position of the detected object on the screen. The interface includes a semicircular radar grid, distance rings, angular reference lines, a rotating sweep line, and a fading detection effect.

The project combines ultrasonic sensing, servo control, Arduino programming, serial communication, mathematical coordinate transformation, and graphical visualization.

2. Objectives

The main objectives of this project were:

1. To build an ultrasonic scanning system using an Arduino UNO.
2. To understand the working principle of the HC-SR04 ultrasonic sensor.
3. To control the angular position of a servo motor using Arduino.
4. To measure the distance of objects at different angular positions.
5. To transmit angle and distance data from Arduino to a computer using serial communication.
6. To develop a radar-style graphical interface using Processing.
7. To understand the conversion of polar coordinates into Cartesian coordinates.
8. To display the measured data in real time.

3. Introduction: The HC-SR04 is an ultrasonic sensor that can be used to determine the distance of an object by measuring the travel time of an ultrasonic pulse. A single sensor normally provides distance information in one direction.

In this project, the HC-SR04 was mounted on a servo motor. By changing the position of the servo and taking distance measurements at different angles, the sensor was used to scan a wider region.

The Arduino sends the measured angle and distance to a computer through serial communication. A Processing program receives this information and represents it graphically as a radar interface.

The project was developed to combine the principles of ultrasonic sensing with servo control, programming, coordinate geometry, and real-time data visualization.

4. Components Used

Component	Quantity	Purpose
Arduino UNO	1	Main controller
HC-SR04 Ultrasonic Sensor	1	Distance measurement
Servo Motor	1	Rotates the ultrasonic sensor
Breadboard	1	Circuit assembly
Jumper Wires	As required	Electrical connections
Computer	1	Runs the Processing visualization

5. Theory and Working Principle

5.1 HC-SR04 Ultrasonic Sensor

The HC-SR04 works on the time-of-flight principle. When a trigger signal is applied, the sensor emits an ultrasonic pulse. The pulse travels towards an object and is reflected back to the sensor.

The Arduino measures the time taken for the pulse to travel to the object and return.

If the distance between the sensor and the object is d , the ultrasonic wave travels a total distance of $2d$. Therefore,

$$d = v \cdot t / 2$$

where:

- d = distance between the sensor and the object
- v = speed of the ultrasonic wave
- t = measured round-trip travel time

The factor of 2 is required because the measured time includes both the outgoing and returning paths of the ultrasonic pulse.

5.2 Servo Scanning

The ultrasonic sensor is mounted on a servo motor. The servo is programmed to rotate through a selected angular range. For each servo position, the Arduino obtains a distance measurement. Therefore, each measurement can be represented by:

(θ, d)

where:

- θ = angular position of the sensor
- d = measured distance

In the implemented system, the servo scans from 0° to 180° and then returns from 180° to 0° .

5.3 Polar to Cartesian Conversion

The measurements obtained from the ultrasonic sensor are naturally represented using polar coordinates because they consist of an angle and a distance.

Processing uses Cartesian coordinates to position objects on the screen. Therefore, the polar coordinates are converted using:

$$\begin{aligned}x &= C_x + r \cos \theta \\y &= C_y - r \sin \theta\end{aligned}$$

- x and y = screen coordinates
- C_x and C_y = centre coordinates of the radar
- r = scaled distance
- θ = measured angle

The negative sign in the equation for y is due to the coordinate system used by Processing, where the positive vertical direction is downward.

5.4 Distance Scaling: The physical distance measured by the HC-SR04 is expressed in centimetres, while the Processing display uses pixels. A scaling factor is therefore used to convert the measured distance into a suitable distance on the screen.

The screen radius is calculated as:

$r = d \times \text{scale}$

where:

- r = distance on the screen in pixels
- d = measured distance in centimetres
- scale = pixels per-centimetre scaling factor

5.5 Serial Communication

The Arduino sends the angle and distance through the serial port in the following form:

Angle, distance

Processing receives the data and separates the angle and distance values before using them to update the radar display.

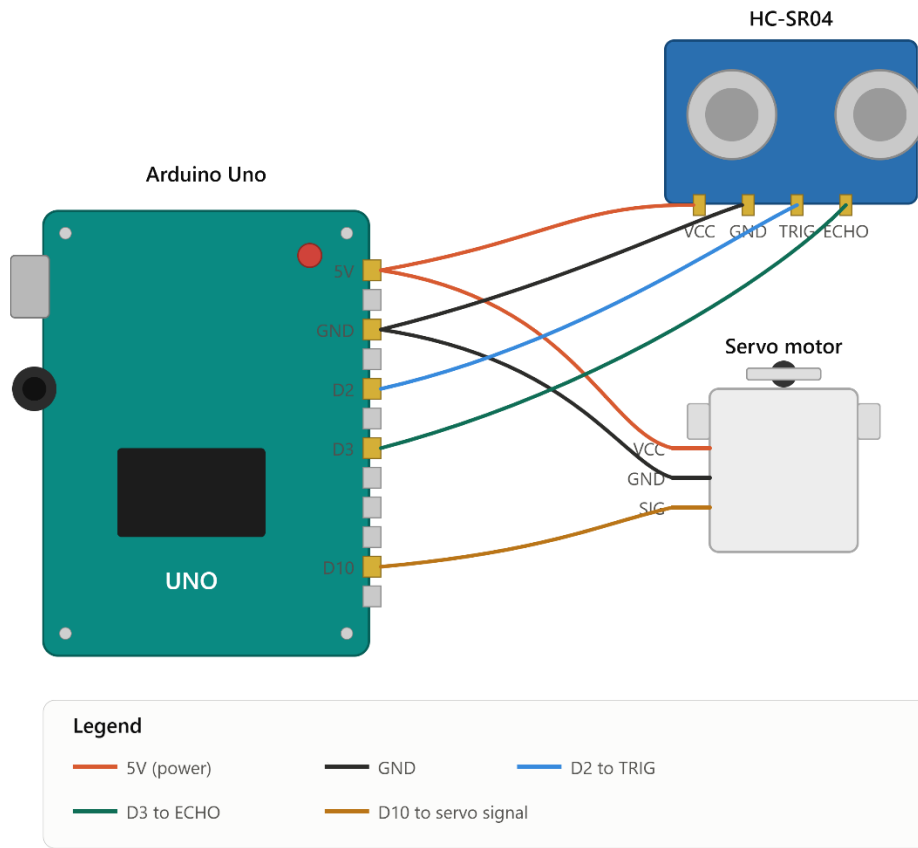
6. Circuit Design

The system consists of an Arduino UNO, an HC-SR04 ultrasonic sensor, and a servo motor.

The trigger and echo pins of the HC-SR04 are connected to digital pins of the Arduino. The servo signal wire is connected to a digital output pin. The Arduino is connected to the computer through USB for programming and serial communication.

Circuit connections:

Component	Arduino Pin
HC-SR04 Trig	D2
HC-SR04 Echo	D3
Servo Signal	D10
HC-SR04 VCC	5V
HC-SR04 GND	GND
Servo VCC	5V
Servo GND	GND



Circuit diagram

7. Experimental Setup and Procedure

The system was assembled using the Arduino UNO, HC-SR04 ultrasonic sensor, servo motor, breadboard, and jumper wires.

The procedure was as follows:

1. The HC-SR04 and servo motor were connected to the Arduino.
2. The Arduino program was uploaded to the board.
3. The servo was programmed to rotate through the required angular range (0 to 180 degrees).
4. The HC-SR04 measured the distance at each servo position.
5. The Arduino sent the angle and distance values through the serial port.
6. Processing was connected to the same serial port.
7. Processing received and separated the angle and distance values.
8. The measured values were converted into Cartesian screen coordinates.

9. The radar grid, distance rings, angular lines, and sweep line were drawn in the Processing interface.

8. Results and Discussion

The completed system successfully scanned objects over the selected angular range and displayed the measurements in a radar-style interface.

The servo controlled the angular position of the HC-SR04, while the ultrasonic sensor provided the distance information. These two values were transmitted to Processing and used to determine the position of the measurement on the display.

The final interface contains:

- A semicircular radar grid
- Concentric distance rings
- Angular reference lines at 30° intervals
- Angular labels
- Distance labels
- A moving green sweep line
- Red detection lines
- A fading persistence effect
- Real-time angle and distance values

The display represents the measurements using polar-to-Cartesian coordinate conversion. The radar center represents the position of the ultrasonic sensor, while the radial distance from the center represents the measured distance of the object.

9. Challenges and Solutions

9.1 Serial Port Access

Processing initially could not access the Arduino's serial port while the Arduino IDE Serial Monitor was open. The issue occurred because the serial port was already being used by the Arduino IDE. The problem was solved by closing the Arduino IDE after uploading the program and then opening the Processing program.

9.2 Coordinate Conversion

The ultrasonic sensor provides measurements as angle and distance, whereas Processing uses screen coordinates. The polar-to-Cartesian equations were therefore used to calculate the position of the measurement on the display.

9.3 Display Scaling

Large measured distances could extend beyond the boundaries of the Processing window. A scaling factor was introduced so that the selected maximum distance could be represented within the radar display.

9.4 Line and Grid Appearance

Different stroke weights were used for the radar grid, sweep line, and detection lines. This allowed the sweep and detection effects to remain visually distinct from the radar grid.

9.5 Persistence Effect

A partially transparent black layer was drawn over the previous frame to gradually fade older graphics. This produced a persistence effect while maintaining the movement of the radar sweep.

10. Limitations

The system has several limitations:

- The HC-SR04 has a limited sensing range.
- The accuracy of detection depends on the position and reflective properties of the object.
- The servo has limited angular resolution and movement speed.
- A single ultrasonic sensor cannot independently identify multiple objects located along the same direction.
- The displayed radar is a graphical representation of ultrasonic measurements and is not an electromagnetic radar system.
- The scanning process takes time because the sensor must move between angular positions.

11. Conclusion

An Arduino-based ultrasonic radar system was successfully developed using an HC-SR04 ultrasonic sensor and a servo motor.

The servo allowed the ultrasonic sensor to scan through a range of angles, while the Arduino measured and transmitted the corresponding distance data. Processing was then used to convert the measurements into Cartesian coordinates and display them through a radar-style interface.

The project provided practical experience in ultrasonic sensing, servo control, Arduino programming, serial communication, coordinate transformation, and graphical programming. It also demonstrated how sensor measurements can be combined with mathematical transformations to create a real-time visualization.

12. Project Evidence

Figure 1. Physical assembly of the ultrasonic radar

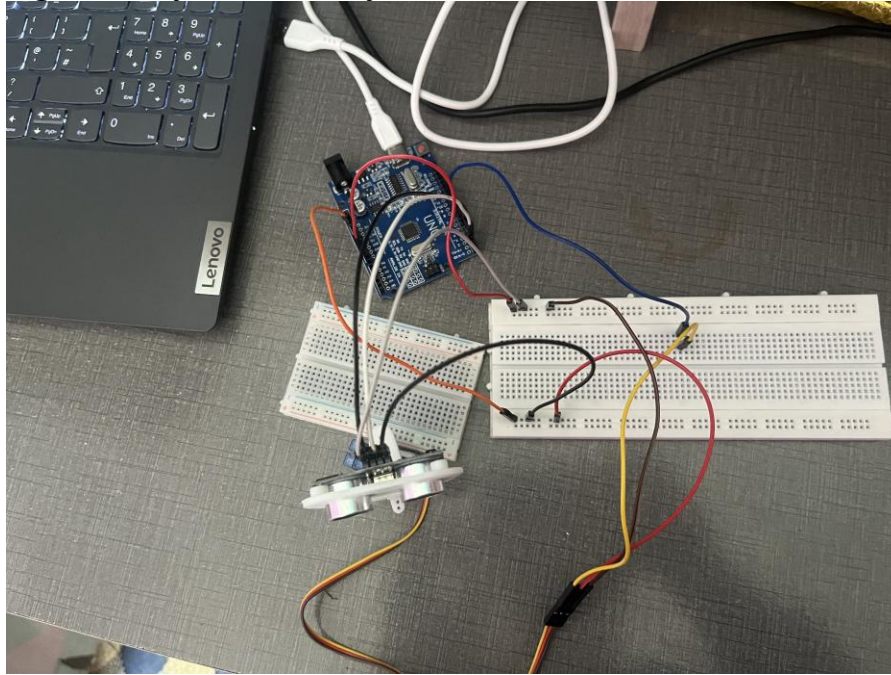


Figure 2. HC-SR04 mounted on the servo motor

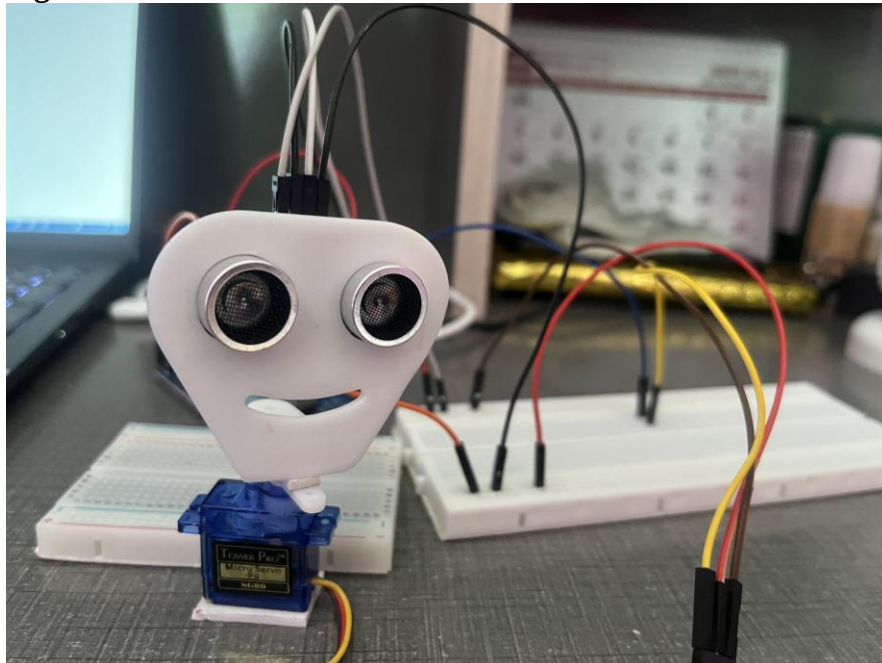


Figure 3. Radar detecting an object

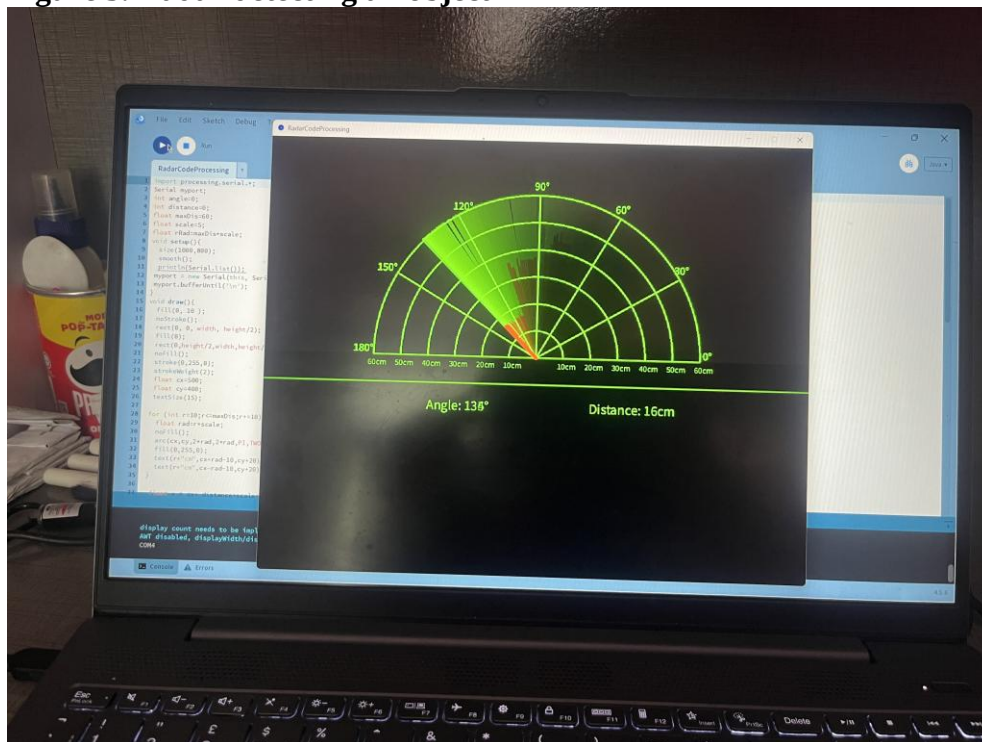
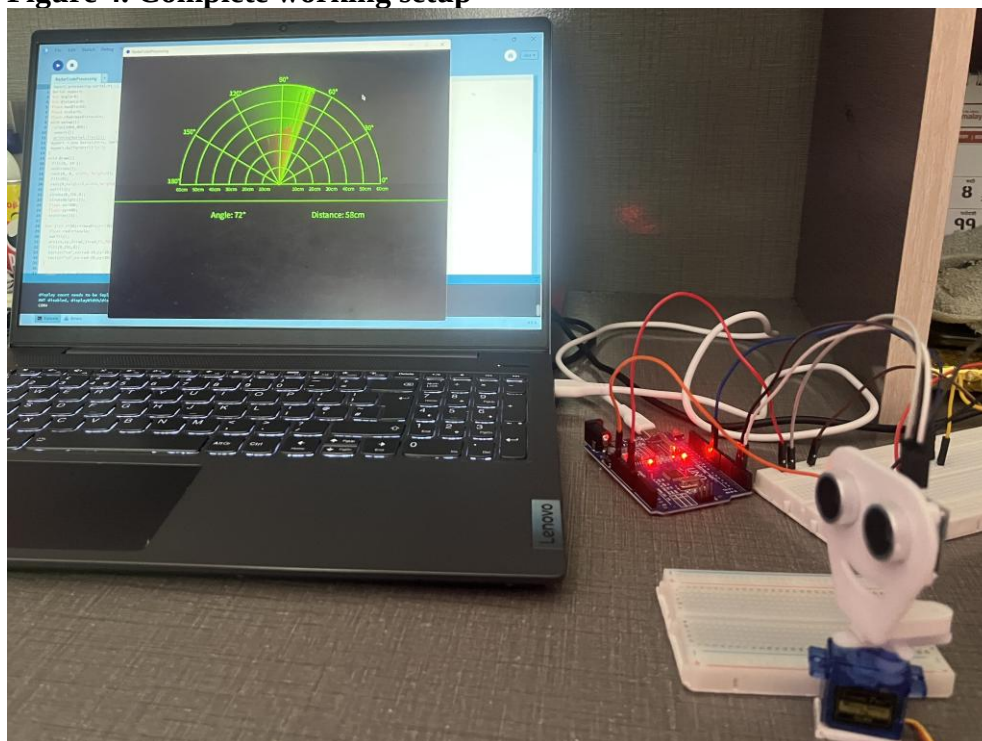


Figure 4. Complete working setup



13. Source Code

Arduino C++

```
#include <Servo.h>

const int trig = 2;
const int echo = 3;

Servo myServo;

void setup() {
    pinMode(trig, OUTPUT);
    pinMode(echo, INPUT);
    myServo.attach(10);
    Serial.begin(9600);
}

void loop() {
    for (int i = 0; i <= 180; i++) {
        myServo.write(i);
        delay(7);
        int distance = calcDist();
        Serial.print(i);
        Serial.print(",");
        Serial.println(distance);
        delay(7);
    }
    for (int i = 180; i >= 0; i--) {
        myServo.write(i);
        delay(7);
        int distance = calcDist();
        Serial.print(i);
        Serial.print(",");
        Serial.println(distance);
        delay(7);
    }
}

int calcDist() {
```

```
digitalWrite(trig, LOW);
delayMicroseconds(2);
digitalWrite(trig, HIGH);
delayMicroseconds(10);
digitalWrite(trig, LOW);
long duration = pulseIn(echo, HIGH);
int distance = duration * 0.034 / 2;
return distance;
}
```

Processing

```
import processing.serial.*;
Serial myport;
int angle=0;
int distance=0;
float maxDis=60;
float scale=5;
float rRad=maxDis*scale;
void setup(){
    size(1000,800);
    smooth();
    println(Serial.list());
    myport = new Serial(this, Serial.list()[0],9600);
    myport.bufferUntil('\n');
}
```

```

void draw(){
    fill(0, 10 );
    noStroke();
    rect(0, 0, width, height/2);
    fill(0);
    rect(0,height/2,width,height/2);
    noFill();
    stroke(0,255,0);
    strokeWeight(2);
    float cx=500;
    float cy=400;
    textSize(15);
    for (int r=10;r<=maxDis;r+=10){
        float rad=r*scale;
        noFill();
        arc(cx,cy,2*rad,2*rad,PI,TWO_PI);
        fill(0,255,0);
        text(r+"cm",cx+rad-10,cy+20);
        text(r+"cm",cx-rad-10,cy+20);
    }

    float x = cx+ distance*scale* cos(radians(angle));
    float y = cy- distance*scale* sin(radians(angle));
    fill(0,255,0);
    stroke(0,255,0);
    line(0,450,1000,450);

    for(int i=0; i<=180;i++){
        if(i%30==0){
            float px=cx+rRad* cos(radians(i));
            float py=cy-rRad* sin(radians(i));
            line(cx,cy, px, py);
            if(i==0){

```

```

    textSize(20);
    text(i+ "°" ,px+5,py-4);
}
else if(i==150||i==180){
    textSize(20);
    text(i+ "°" ,px-37,py-4);
}else{
    textSize(20);
    text(i+ "°" ,px-10,py-10);
}}}
textSize(25);
text("Angle: "+angle+"°",300,500);
text("Distance: "+distance+"cm",600,500);
float swpx= cx+ rRad* cos(radians(angle));
float swpy= cy- rRad* sin(radians(angle));
stroke(0,255,0);
strokeWeight(7);
line(cx,cy,swpx,swpy);
if (distance > 0 && distance <= maxDis) {
    float intensity = map(distance, 0, maxDis, 255, 80);
    stroke(255, 0, 0, intensity);
    strokeWeight(7);
    line(cx, cy, x, y);
}}
void serialEvent(Serial myport){
    String data = myport.readStringUntil('\n');
    if(data!=null){
        data = trim(data);
        String[] values=split(data,',');
        angle = int(values[0]);
        distance= int(values[1]);
    }}

```

A project by:
Ritiz Shrestha